Sztuczna Inteligencja i Inżynieria Wiedzy

Zadanie 1 – Raport Implementacyjny

2026

Spis treści

1 Struktura projektu **2**

2 Zadanie 1 – Wyszukiwanie najkrótszej ścieżki **2**

2.1 Algorytm Dijkstry (1a) 2

2.2 Algorytm A* – kryterium czasu (1b) 2

2.3 Algorytm A* – kryterium przesiadek (1c) 3

2.4 Modyfikacje przyspieszające obliczenia (1d) 3

2.5 Interfejs CLI – Zadanie 1 3

3 Zadanie 2 – Problem komiwojażera (Tabu Search) **4**

3.1 Interfejs CLI – Zadanie 2 4

3.2 Tabu Search bez ograniczenia T (2a) 4

3.3 Tabu Search ze zmiennym rozmiarem T (2b) 4

3.4 Kryterium aspiracji (2c) 4

3.5 Próbkowanie sąsiedztwa (2d) 5

3.6 Inicjalizacja trasy i funkcja kosztu 5

3.7 Ewaluacja trasy 5

4 Wczytywanie danych GTFS i budowanie grafu **5**

5 Biblioteki zewnętrzne **6**

6 Napotkane problemy **6**

1 Struktura projektu

Projekt realizuje wyszukiwanie połączeń kolejowych na danych GTFS Kolei Dolnośląskich. Kod źródłowy zorganizowany jest w pakiecie `src/` z podziałem na moduły:

```
src/
  config.py              # stale konfiguracyjne
  algorithms/
    astar.py             # algorytm A*
    dijkstra.py          # algorytm Dijkstry
    tabu_search.py       # Tabu Search (warianty 2a-2d)
    common.py            # wspolne funkcje (relaksacja krawedzi,
       rekonstrukcja sciezki)
  gtfs/
    loader.py            # wczytywanie plikow GTFS
    graph.py             # budowanie grafu polaczen
    calendar.py          # filtrowanie kursow wg daty
  cli/
    pathfinder_cli.py    # interfejs CLI -- Zadanie 1
    tsp_cli.py           # interfejs CLI -- Zadanie 2
  utils/
    time_utils.py        # konwersje czasu
    geo.py               # odleglosc haversine
server.py                # serwer HTTP z API REST
gtfs_visualizer.html     # interfejs webowy
```

2 Zadanie 1 – Wyszukiwanie najkrótszej ścieżki

2.1 Algorytm Dijkstry (1a)

Plik: `src/algorithms/dijkstra.py`
Funkcja: `dijkstra(graph, starts, ends, criterion)`

Implementacja klasycznego algorytmu Dijkstry z kolejką priorytetową (`heapq`). Obsługuje oba kryteria optymalizacji: czas przejazdu (`t`) oraz liczbę przesiadek (`p`).

Uruchamiany z poziomu CLI flagą `-dijkstra`:

```
python -m src.cli.pathfinder_cli "Wroclaw␣Glowny" "Jelenia␣Gora" t
   \
    "2026-03-09␣08:00" --dijkstra
```

2.2 Algorytm A* – kryterium czasu (1b)

Plik: `src/algorithms/astar.py`
Funkcja główna: `astar(graph, starts, ends, criterion, ...)`
Heurystyka: `heuristic_time(node, end_stops, gtfs_data)`

Heurystyka czasowa szacuje pozostały koszt jako odległość geograficzną (wzór haversine, funkcja `distance_km` w `src/utils/geo.py`) podzieloną przez maksymalną prędkość po-

ciągu (`MAX_SPEED_KMH` w `src/config.py`). Gwarantuje to dopuszczalność heurystyki – nigdy nie przeszacowuje rzeczywistego czasu.

2.3 Algorytm A* – kryterium przesiadek (1c)

Plik: `src/algorithms/astar.py`
Heurystyka: `heuristic_transfers(node, end_stops, direct_set)`
Funkcja pomocnicza: `build_direct_stop_set(graph, end_stops)`

Heurystyka przesiadkowa zwraca 0, jeśli istnieje bezpośrednie połączenie z bieżącego węzła do stacji docelowej (precomputed `direct_set`), a w przeciwnym razie zwraca 1. Jest to heurystyka dopuszczalna – nigdy nie zawyża liczby przesiadek.

2.4 Modyfikacje przyspieszające obliczenia (1d)

Plik: `src/algorithms/astar.py`

Zaimplementowane usprawnienia:

- **Wczesne zatrzymanie** (*early exit*): algorytm kończy działanie natychmiast po wyciągnięciu węzła docelowego z kolejki, bez eksploracji pozostałych gałęzi.
- **Grupowanie przystanków stacji** (`station_group` w `src/gtfs/graph.py`): zamiast jednego węzła startowego, do kolejki dodawane są wszystkie perony danej stacji, co eliminuje zbędne krawędzie transferowe na początku trasy.
- **Pominięcie zdominowanych stanów**: jeśli węzeł zostaje wyciągnięty z kolejki z kosztem wyższym niż dotychczas znane optimum dla danego przystanku, jest pomijany (`if cost > dist[node]: continue`).
- **Precomputed** `direct_set` dla heurystyki przesiadkowej: jednorazowo obliczany w `build_direct_stop_set()` i przekazywany jako parametr, zamiast obliczać przy każdym wywołaniu heurystyki.

2.5 Interfejs CLI – Zadanie 1

Plik: `src/cli/pathfinder_cli.py`
Funkcja: `main()` / `_run()`
Formatowanie wyników: `print_results()`

```
# Przyklad uzycia (A*, kryterium czasu):
python -m src.cli.pathfinder_cli "Wroclaw␣Glowny" "Legnica" t "
   2026-03-10␣07:30"

# Dijkstra, kryterium przesiadek:
python -m src.cli.pathfinder_cli "Swidnica␣Miasto" "Walbrzych␣
   Glowny" p \
    "2026-03-10␣09:00" --dijkstra
```

Na standardowe wyjście wypisywane są kolejne odcinki trasy (przystanek początkowy, końcowy, linia, czas odjazdu/przyjazdu). Na standardowe wyjście błędów – wartość zminimalizowanego kryterium i czas obliczeń.

3 Zadanie 2 – Problem komiwojażera (Tabu Search)

3.1 Interfejs CLI – Zadanie 2

Plik: `src/cli/tsp_cli.py`
Funkcja: `main()`

```
# Przyklad uzycia (Tabu Search z aspiracja i probkowaniem):
python -m src.cli.tsp_cli "Wroclaw␣Glowny" "Legnica;Jelenia␣Gora;
   Swidnica" \
    t "2026-03-10␣08:00" --2c --2d
```

Flagi wyboru wariantu Tabu Search: `-2a`, `-2b` (domyślnie), `-2c` (aspiracja), `-2d` (próbkowanie sąsiedztwa).

Macierz kosztów

Funkcja: `compute_cost_matrix()` – oblicza macierz kosztów przejazdu między wszystkimi parami przystanków za pomocą A* (optymistyczna wersja bez uwzględniania czasu przyjazdu).
Funkcja: `compute_cost_matrix_chained()` – iteracyjne przeliczanie kosztów z rzeczywistymi czasami przyjazdów (udoskonalenie wyniku Tabu Search).

3.2 Tabu Search bez ograniczenia T (2a)

Plik: `src/algorithms/tabu_search.py`
Funkcja: `tabu_search_unlimited(cost_matrix, n, criterion)`

Podstawowy algorytm Tabu Search zgodny z algorytmem Knoxa. Lista tabu `T` rośnie nieograniczenie przez cały czas działania. Ruchem jest zamiana dwóch przystanków w trasie (*2-opt reversal*).

3.3 Tabu Search ze zmiennym rozmiarem T (2b)

Plik: `src/algorithms/tabu_search.py`
Funkcja: `tabu_search_variable_tenure(cost_matrix, n, criterion)`

Rozmiar kolejki FIFO tablicy tabu wynosi $t = \lceil |L|/2 \rceil$, gdzie $|L|$ to liczba przystanków pośrednich. Dla małych tras lista jest krótka (mniejsze zużycie pamięci), dla długich – dłuższa (lepsza ochrona przed cyklami).

3.4 Kryterium aspiracji (2c)

Plik: `src/algorithms/tabu_search.py`
Funkcja: `tabu_search_core(..., use_aspiration=True)`

Aktywowane flagą `use_aspiration=True` w `tabu_search_core()`. Kryterium aspiracji: jeśli ruch jest na liście tabu, ale prowadzi do rozwiązania *lepszego* niż dotychczasowe najlepsze `best_cost`, jest dopuszczany.

3.5 Próbkowanie sąsiedztwa (2d)

Plik: `src/algorithms/tabu_search.py`
Funkcja: `tabu_search_core(..., use_sampling=True)`

Aktywowane flagą `use_sampling=True`. Zamiast przeszukiwać wszystkie $O(n^2)$ sąsiadów 2-opt, losowo próbkowana jest podzbiorowa liczba kandydatów (`TABU_SAMPLE_SIZE` z `src/config.py`). Skraca czas iteracji kosztem ewentualnej gorszej jakości rozwiązania.

3.6 Inicjalizacja trasy i funkcja kosztu

Plik: `src/algorithms/tabu_search.py`
Funkcje:

- `nearest_neighbour_init()` – zachłanna inicjalizacja trasą metodą najbliższego sąsiada,
- `random_init()` – losowa permutacja przystanków,
- `tour_cost_matrix()` – oblicza całkowity koszt trasy na podstawie macierzy kosztów.

3.7 Ewaluacja trasy

Plik: `src/cli/tsp_cli.py`
Funkcja: `evaluate_tour()`

Po znalezieniu optymalnej kolejności przystanków, `evaluate_tour()` symuluje rzeczywistą podróż z uwzględnieniem rzeczywistych czasów odjazdu/przyjazdu, kursów oraz przesiadek. Obsługuje też bezpośrednie przejazdy (`check_direct_connections`, `find_continuation`).

4 Wczytywanie danych GTFS i budowanie grafu

Plik: `src/gtfs/loader.py`
Funkcja: `load_gtfs(path)` – wczytuje wszystkie pliki GTFS do słowników Pythona.

Plik: `src/gtfs/graph.py`
Funkcja: `build_graph(gtfs_data, start_date, num_days, ...)` – buduje skierowany graf zależny od czasu:

- krawędzie kursów ze `stop_times.txt` (dla aktywnych serwisów),
- krawędzie transferowe między przystankami tej samej stacji (`_add_transfer_edges()`).

Plik: `src/gtfs/calendar.py`
Funkcja: `is_service_active(service_id, date, ...)` – sprawdza, czy dany kurs jest aktywny w podanej dacie, uwzględniając wyjątki z `calendar_dates.txt`.

Wyszukiwanie przystanków po nazwie: `find_stop()` w `src/gtfs/graph.py` – dopasowanie dokładne, z sufiksem„główny”, prefiksowe, a następnie podciąg nazwy.

5 Biblioteki zewnętrzne

- **Python 3.x** – język implementacji.
- **heapq** (biblioteka standardowa) – kolejka priorytetowa w algorytmach Dijkstry i A*.
- **csv** (biblioteka standardowa) – wczytywanie plików GTFS.
- **datetime** (biblioteka standardowa) – obsługa dat i kalendarza kursów.
- **math** (biblioteka standardowa) – obliczenia trygonometryczne (haversine).
- **random** (biblioteka standardowa) – losowe próbkowanie sąsiedztwa w Tabu Search.
- **http.server** (biblioteka standardowa) – serwer HTTP (`server.py`).
- **itertools** (biblioteka standardowa) – generowanie permutacji (brute force dla $n \leq 4$ w `tsp_cli.py`).

Projekt nie korzysta z zewnętrznych bibliotek spoza biblioteki standardowej Pythona.

6 Napotkane problemy

- **Czasy przekraczające 24h w GTFS** – plik `stop_times.txt` zawiera czasy w formacie `HH:MM:SS`, gdzie `HH` może przekraczać 23 (np. `25:10:00` oznacza 1:10 następnego dnia). Rozwiązano to przez konwersję wszystkich czasów do sekund od północy (`time_to_sec()` w `src/utils/time_utils.py`) i obsługę wartości > 86400.
- **Hierarchia stacji w GTFS** – przystanki mogą być peronami (podrzędnymi) lub stacjami (nadrzędnymi). Krawędzie transferowe muszą łączyć *wszystkie* perony tej samej stacji, nie tylko pary o identycznym `stop_id`. Obsłużono to w `_add_transfer_edges()`.
- **Dopuszczalność heurystyki A*** – heurystyka musi być dopuszczalna (nie przeszacowywać). Dla kryterium czasu użyto odległości geograficznej podzielonej przez `MAX_SPEED_KMH`. Dla kryterium przesiadek wartość 0/1 jest zawsze dopuszczalna.
- **Macierz kosztów a rzeczywiste czasy** – optymistyczna macierz kosztów (A* bez uwzględniania łańcucha czasu) może być nieadekwatna po ustaleniu kolejności przystanków. Rozwiązano to przez iteracyjne przeliczanie macierzy z rzeczywistymi czasami przyjazdów (`compute_cost_matrix_chained()`).
- **Identyfikacja przesiadek** – zmiana `trip_id` nie zawsze oznacza przesiadkę (ten sam pojazd może kontynuować kurs pod innym identyfikatorem `block_id`). Obsłużono to w `find_continuation()` i `check_direct_connections()`.